



Amlogic Android Deployment Guide

Version: v0.1

Release Date: 2026-06-25

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称“Amlogic”）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

 **Amlogic**，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址: www.amlogic.cn
- 售前咨询: contact@amlogic.com
- 技术支持: support@amlogic.com

修订记录

版本	修改说明
v0.1 (2026-06-25)	初始版本

目录

1. 部署概述

- 1.1 NNAPI 接入方案
- 1.2 LiteRT (AML Delegate) 接入方案
- 1.3 JNI (NNSDK) 接入方案
- 1.4 Android 版本与部署方式支持状态
 - 1.4.1 部署方式支持
 - 1.4.2 平台模型支持
- 1.5 三种部署方案对比

2. NNAPI 部署方案

- 2.1 流程图
- 2.2 环境确认
- 2.3 快速确认
- 2.4 NNAPI 部署

3. LiteRT部署方案

- 3.1 流程图
- 3.2 部署步骤
 - 3.2.1 部署环境确认
 - 3.2.2 模型准备
 - 3.2.3 Interpreter API 部署
 - 3.2.4 CompiledModel API 部署

4. JNI 部署方案

- 4.1 流程图
- 4.2 参考工程 (MobileNet Demo)
 - 4.2.1 工程结构
 - 4.2.2 分层设计
 - 4.2.3 JNI 封装要点
 - 4.2.4 Kotlin API 一览
- 4.3 部署步骤
 - 4.3.1 环境与版本确认
 - 4.3.2 准备 Native 库
 - 4.3.3 编译 JNI 桥接库
 - 4.3.4 准备模型与测试资源
 - 4.3.5 推理流程
 - 4.3.6 自主封装指南
 - 4.3.7 编译与运行

1. 部署概述

在Android平台将模型调度至Amlogic NPU完成硬件加速，共提供**NNAPI**、**LiteRT(AML Delegate)**、**JNI(NNSDK)**三类成熟的接入方案。三类方案在调用链路长度、集成接口门槛、支持模型格式、底层可控度上各有明确差异，选型时可结合目标设备Android系统版本、产品形态(通用第三方APK/方案机预制ROM/OTT定制系统)、性能与开发效率诉求综合判断。

1.1 NNAPI 接入方案

该方案在应用侧完全复用LiteRT/TFLite标准开发接口，仅需通过 `SetNNAPI(True)` 一行配置即可开启NNAPI硬件加速路径。在Amlogic平台的完整调度链路为：算子请求经NNAPI下发后，依次通过 `nnservice`、`nnrt`、`nnsdk`层最终调度至ADLA驱动，完成NPU硬件执行。此接入方式无需在APK包内集成AML Delegate相关依赖，大幅缩减应用包体积，但有明确前置约束：

- 目标设备ROM必须已预置并正常启动 `nnservice` 服务，
- `nnservice`、`nnrt`、`nnsdk`、底层驱动全链路版本完全匹配，避免出现兼容性问题

1.2 LiteRT (AML Delegate) 接入方案

该方案以LiteRT作为应用侧核心推理运行时，通过挂载专属**AML Delegate**组件，自动将.tflite模型中所有支持NPU加速的子图拆分下沉至Amlogic NPU执行，完整调度链路为：**AML Delegate** → **底层 Runtime** → **NNSDK** → **ADLA Driver**，直接访问NPU硬件资源。应用侧仍沿用此前已适配的Interpreter或CompiledModel标准API，仅需新增Delegate配置项与对应Native依赖声明即可完成NPU加速启用。

- 和NNAPI方案相比，Delegate模式的调用链路更短，可在编译阶段提前完成子图智能划分、算子融合、推理缓存策略等平台级深度优化；
- 和JNI方案相比，完全保留LiteRT标准开发范式，可无缝对接Google AI Edge全链路工具链，降低跨项目迁移成本。

适用场景：以LiteRT为核心推理框架、需要深度适配Amlogic NPU能力的APK或Native方案。

1.3 JNI (NNSDK) 接入方案

该方案直接以Amlogic原生NNSDK作为底层推理引擎，Java/Kotlin层业务代码通过自定义JNI接口调用底层Native库，由NNSDK直接对接ADLA完成全流程NPU推理，完全绕过LiteRT与NNAPI两层抽象封装。客户可根据自身工程架构需求，自主封装NNSDK接口，定制专属JNI适配层。该方案是三类接入方式中调用链路最短的模式，推理性能可做到极致释放，非常适配方案机、OTT定制系统等需要深度定制、对底层硬件资源有强管控需求的交付场景；但该方案与设备BSP/DDK版本强绑定，跨设备复用能力仅覆盖同系列Amlogic芯片平台。

适用场景：芯片方案级交付项目、需要完全绕过LiteRT/NNAPI抽象层、对NPU性能做极致调优，或需要适配特殊自定义模型格式的项目。

1.4 Android 版本与部署方式支持状态

1.4.1 部署方式支持

Android 版本	NNAPI	LiteRT	JNI
Android R	✓	✓	✓

Android 版本	NNAPI	LiteRT	JNI
Android T	✓	✓	✓
Android U	✓	✓	✓
Android 16	✓	✓	✓

1.4.2 平台模型支持

平台	Float32 模型	量化模型
A311Y3	✓	✓
其他平台	×	✓

A311Y3 同时支持 Float32 与量化模型；**其他平台** 仅支持量化模型（如 Int8 / UInt8/int16 等）。

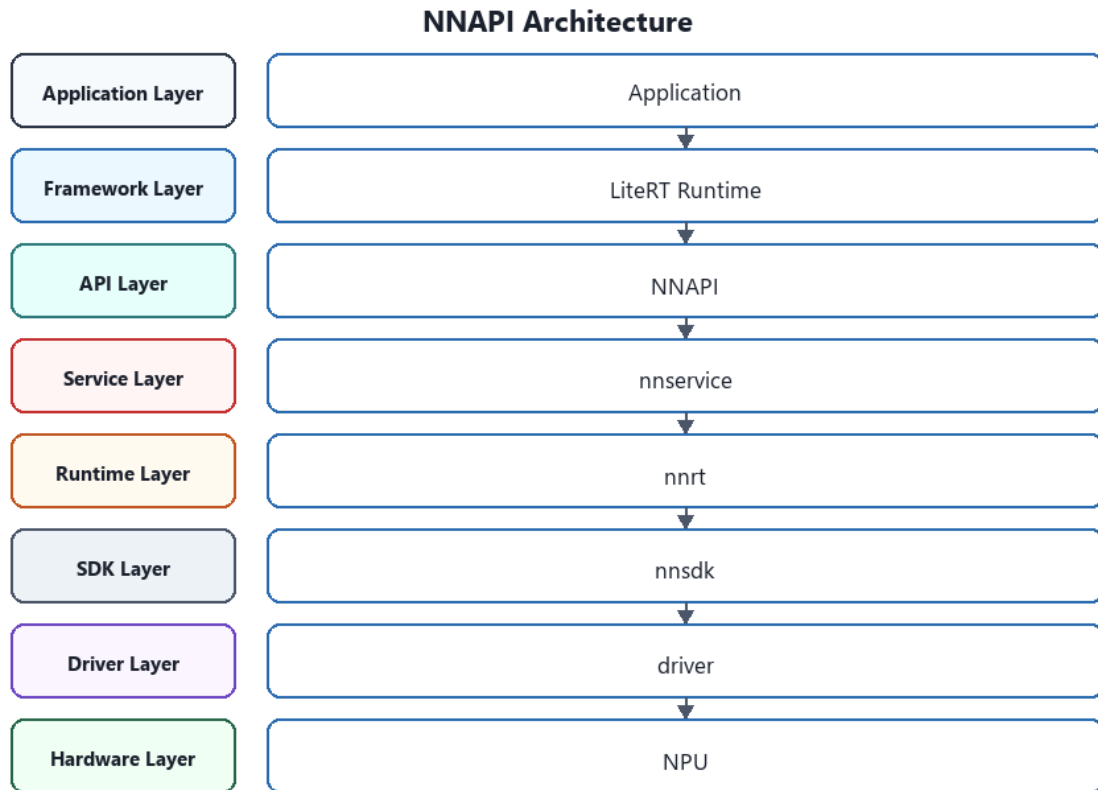
1.5 三种部署方案对比

对比维度	NNAPI	LiteRT	JNI
标准归属	LiteRT NNAPI 接入范式 + Amlogic Runtime/Service 实现	Google LiteRT 扩展机制 + AML Delegate 实现	Amlogic 自研 NNSDK, 私有栈
模型格式	标准 .tflite 文件	标准 .tflite 文件	Amlogic专属 .ad1a 格式
算子覆盖能力	受NNAPI通用算子集与全链路版本实现限制，部分算子自动回退CPU	由 AML Delegate 决定，可针对平台深度优化/融合	由Amlogic NNSDK完全自定义，支持平台级专属算子定制
图优化与融合	由 Runtime/Service 侧编译链路决定，应用侧可控性较弱	可在 Delegate 编译阶段做子图划分、融合、量化策略等	由 NNSDK 编译链路决定，方案侧可控

对比维度	NNAPI	LiteRT	JNI
性能与调优	受 nnservice 调度、driver 策略及多进程共享影响	链路更短，便于做缓存、定点策略、平台专项优化	链路最短，易做极致优化，需自行维护全栈
兼容与升级	与 LiteRT 版本及 nnservice/nnrt/nnsdk/driver 版本矩阵强相关	与 LiteRT 版本 + Delegate 版本相关，与 NNAPI 解耦	与 BSP/DDK/芯片代际强绑定
集成复杂度	应用侧较简单，依赖 ROM 预置 Runtime/Service	需集成 AAR/.so、配置 Delegate 参数，Native 依赖较多	需维护 JNI、ABI、模型路径与版本绑定
典型优势	应用侧 API 标准、集成改动小、沿用 TFLite/LiteRT 范式	性能与可控性高、可深度定制、便于平台级优化	灵活定制、方案级交付、对底层资源掌控力强
典型劣势	依赖 nnservice 常驻、链路较长、问题定位需跨 Runtime/Service/Driver	需维护 LiteRT 与 Delegate 版本矩阵	应用层标准化弱、跨生态复用难、学习曲线陡
适用场景	老版本 ROM、已预置 nnrt/nnservice 的方案、希望最小化应用改造	方案机/OTT/定制 ROM、追求 NPU 性能与可控性	方案级交付、需绕过 LiteRT/NNAPI 的定制场景

2. NNAPI 部署方案

2.1 流程图



2.2 环境确认

部署前需确认设备侧 **nnservice**、**nnrt** 与 **driver / nnsdk** 已安装且版本配套，避免 Runtime、Service 与驱动不匹配导致 NNAPI 接入方案不可用。

- **nnservice 状态确认**

确认系统侧 NN Service 已安装并正常运行，且 `android.hardware.neuralnetworks-service-aidl-driver` 已成功挂载，执行如下命令。

```
ps -A | grep neuralnetworks
```

- **预期结果**：结果中显示有neuralnetworks

- **nnrt 版本确认**

执行如下命令

```
strings /vendor/lib64/libadla_nnrt.so | grep VERSION
```

- **预期结果**：返回字符串中包含编译器版本信息，格式为 `match_nnsdk_v3_compiler_vX.Y.Z`（例如 `match_nnsdk_v3_compiler_v3.4.0`），提取出的版本号 `X.Y.Z` 须 $\geq$ `v3.4.0`

- **NNSDK 版本确认**

```
strings /vendor/lib64/libnnsdk.so | grep NNSDK
```

- **预期结果:**

- **NNSDK:** `NNSDK Version: v3.0.0, 2026.05`, 对NNSDK1版本, 版本号>=v3.0.0
- **NNSDK2:** `NNSDK2 Version: v1.0.0, 2026.05`, 对NNSDK2版本, 版本号>=v1.0.0

2.3 快速确认

使用 Google 提供的 **benchmark 二进制** (`64位 android_aarch64_benchmark_model`) 快速验证 NNAPI 路径是否可用。

下载链接: [Android benchmark model](#)

- **运行 benchmark**

```
#64bit系统
./android_aarch64_benchmark_model --graph=mobilenet_v2_1.0_224_quant.tflite
--use_nnapi=true

#32位系统
./android_arm_benchmark_model --graph=mobilenet_v2_1.0_224_quant.tflite --
use_nnapi=true
```

- **预期结果:** 命令成功运行, 无报错退出。推理过程中 ADLA 中断计数有增加

- **NPU中断确认**

推理前后分别执行如下命令:

```
cat /proc/interrupts | grep adla
```

- **预期结果:** 第二列对应的中断数有增加。

```
/data/Ci_LiteRT_Interpreter_case # cat /proc/interrupts | grep adla
91:          514          0          0          0          0          0
  0           0  GICv3 292 Level    adla
```

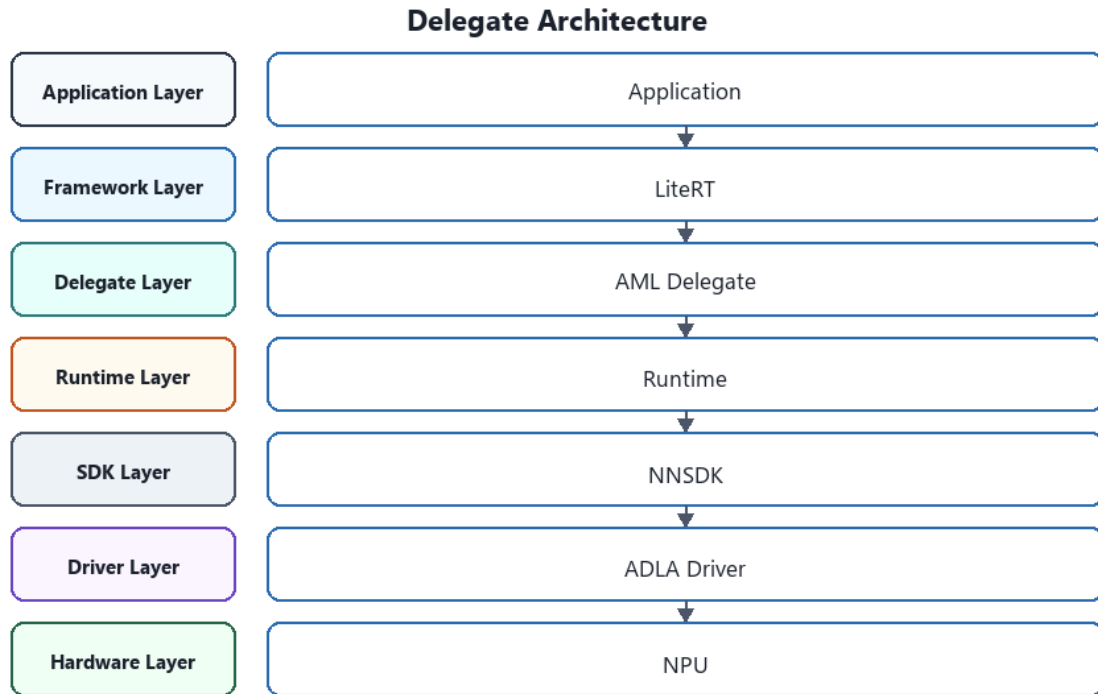
2.4 NNAPI 部署

- **模型准备:** 准备平台支持的 `.tflite` 模型。NNAPI 部署时, A311Y3 支持 float 及 int8/uint8 量化模型, 其余平台仅支持 int8/uint8 量化模型。
- **部署参考:** 参考[image_classification](#) 配置NNAPI

3. LiteRT部署方案

LiteRT有Interpreter API 部署方案和CompiledModel API 部署方案。

3.1 流程图



3.2 部署步骤

3.2.1 部署环境确认

- 基础环境确认

```
ls /dev/ad1a0
ls /vendor/lib64/libnnsdk.so
ls /vendor/lib64/libadla_nnrt.so
ls /vendor/lib64/libcompiler_adla.so
```

- 预期结果：以上命令均执行成功，设备节点 `/dev/ad1a0` 及相关 `.so` 库文件均存在。

- NNSDK版本确认

```
strings /vendor/lib64/libnnsdk.so | grep NNSDK
```

- 预期结果：

NNSDK: `NNSDK Version: v3.0.0, 2026.05` ,对NNSDK1版本, 版本号>=v3.0.0

NNSDK2: `NNSDK2 Version: v1.0.0, 2026.05` ,对NNSDK2版本, 版本号>=v1.0.0

- Adla Version确认

```
cat /sys/class/adla/ad1a0/device/kmd_version
```

- **预期结果:** ADLA Version: 2.0.2.0.0
- **AAR Package版本**
 - 下载的aar package版本号>=3.0.0.3。aar package 下载地址: [aar_package_link](#)

3.2.2 模型准备

- **准备模型:** 准备好与 AML Delegate 支持算子/量化策略一致的 `.tflite` 模型产物
- **功能/精度回归验证:** 需要时对模型输出做一次 **功能/精度回归** (CPU 回退 vs NPU 输出), 确认没有因量化或算子差异引入明显偏差
- **编译/加载缓存注意:** 若当前方案使用 **编译/加载缓存**, 提前确认缓存目录与失效规则 (避免“旧缓存 + 新模型”造成的异常)。

3.2.3 Interpreter API 部署

- **Interpreter API 方案快速验证**
 - **下载链接:** [Interpreter Test Bin](#)
 - **环境准备:** 将已有的 `aml_delegate_test_bin` 推到设备可执行目录 (例如 `/data/local/tmp/` 或你们约定的工作目录), 并保证权限可执行:
 - 先 `adb push` 二进制文件到目标目录
 - 再在设备上执行 `chmod +x <binary>`
 - **运行程序:** `./aml_delegate_test_bin mobilenet_v2_1.0_224_quant.tflite`
 - **运行结果:**

预期输出类似如下log:

```
The minimum amount of memory needed for initializing the context is:
    model alloc from os          0x00000000,( 0.00 MByte)
    model alloc from drv         0x0085ae8c,( 8.36 MByte)
    model addition alloc from os 0x00031700,( 0.19 MByte)
sys mem_available 6004556 KB (5863.82 MB)
Evaluate the smmu_tlb_type is 0
[ADLAU WARN] [evaluate_addition_mem_usage:5185]
Your model file may contain unnecessary data, which will cause waste of
memory.
It is recommended to check your settings when generate the model file.
memory usage:
    model alloc from os          0x00000000,( 0.00 MByte)
    model alloc from drv         0x0085ae8c,( 8.36 MByte)
    model addition alloc from os 0x0004b2c0,( 0.29 MByte)
5.AML_Delegate AllocateTensors
6.AML_Delegate set input
6.AML_Delegate Start to do invoke
Inference status: 0
Output tensor 0
Output tensor name: Identity
Output tensor type: 3
Output tensor size (bytes): 10140
Output tensor dims: 1 2535 4
Output tensor 1
Output tensor name: Identity_1
Output tensor type: 3
Output tensor size (bytes): 202800
Output tensor dims: 1 2535 80
```

```
----- finished -----  
7.CI test exit (skip interpreter teardown to avoid hang)
```

- **AAR 集成**

- 将 `LiteRT_AML.aar` 放入工程，归档到模块 `libs/` 目录（或按你项目约定放到对应位置）。AML Delegate与 LiteRT 详细内容请参考 [07 Amlogic NPU LiteRT.pdf](#)

3.2.4 CompiledModel API 部署

- **CompiledModel API 方案快速验证:**

- 下载链接: [CompiledModel Test Bin](#)
- **环境准备:** 将resource push 至 `/data/LiteRT/`，并在执行前设置 `LD_LIBRARY_PATH`（至少包含库目录与本目录）`export`
`LD_LIBRARY_PATH=/vendor/lib64:/data/LiteRT:$LD_LIBRARY_PATH`
- **运行程序:** `./aml_CompiledModel_benchmark mobilenet_v2_1.0_224_quant.tflite`
- **运行结果:**

预期输出类似如下log:

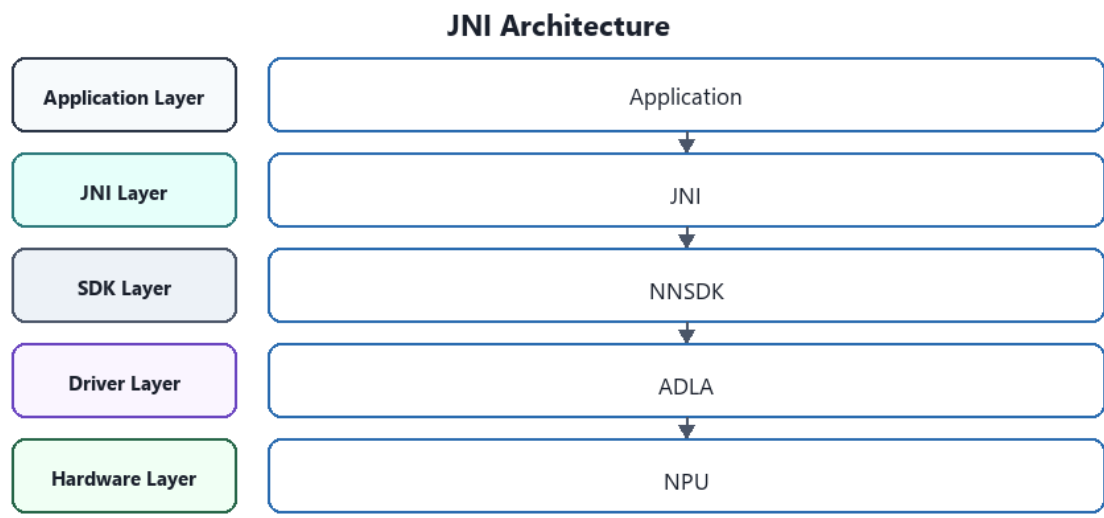
```
-----[AML GetInputRequirements] Enter -----  
-----[AML GetOutputRequirements] Enter -----  
-----[AML GetOutputRequirements] Enter -----  
-----[AML GetOutputRequirements] Enter -----  
***** Create CompiledModel Finished  
---  
Model signature count: 1  
signature[0] = serving_default  
-----创建输入buffer-----  
-----创建输出buffer-----  
-----随机填充输入数据-----  
Running inference...  
-----[AML RegisterTensorBuffer] Enter -----  
-----[AML RegisterTensorBuffer] Enter -----  
-----[AML RegisterTensorBuffer] Enter -----  
-----[AML RegisterTensorBuffer] Enter -----  
LiteRtDispatchInvocationContextT::AttachInputBuffer  
-----[AML GetTensorBuffer] Enter -----  
LiteRtDispatchInvocationContextT::AttachOutputBuffer  
-----[AML GetTensorBuffer] Enter -----  
LiteRtDispatchInvocationContextT::AttachOutputBuffer  
-----[AML GetTensorBuffer] Enter -----  
LiteRtDispatchInvocationContextT::AttachOutputBuffer  
-----[AML GetTensorBuffer] Enter -----  
InvocationContext Execute  
-----推理完成-----  
output[0] size=97344  
output[1] size=389376  
output[2] size=24336  
-----[AML GetTensorBuffer] Enter -----  
-----[AML GetTensorBuffer] Enter -----  
-----[AML GetTensorBuffer] Enter -----  
-----[AML GetTensorBuffer] Enter -----
```

```
INFO: [accelerator_registry.cc:41] DestroyAccelerator:  
ptr=0xb40000772280a6e0, name=CpuAccelerator  
INFO: [accelerator_registry.cc:41] DestroyAccelerator:  
ptr=0xb40000772280a680, name=NpuAccelerator
```

- **AAR 集成:** (TODO)
 - CompiledModel API方案对应的AAR package待Release。

4. JNI 部署方案

4.1 流程图



4.2 参考工程 (MobileNet Demo)

仓库内提供可运行的 JNI 参考工程 <https://github.com/Amlogic-NN/amlnn-model-playground/tree/main/examples/mobilenet/jni>，演示如何将 **NNSDK2** 以 JNI方案接入 Android APK，并完成 MobileNetV2 量化模型的图像分类推理。

4.2.1 工程结构

```
jni/
├─ app/
│   ├── CMakeLists.txt           # 编译 libnnsdk_jni.so
│   ├── build.gradle.kts         # ABI、jniLibs、CMake 配置
│   └─ src/main/
│       ├── assets/demo/        # 模型、标签、测试图片
│       │   ├── mobilenet_v2_1.0_224_quant.adla
│       │   ├── labels.txt
│       │   └─ *.jpg / *.jpeg
│       ├── java/com/example/NN_JNI/
│       │   ├── NnSdk2.kt        # NNSDK2 Kotlin 封装
│       │   ├── NnStruct.kt      # 结构体读写辅助
│       │   ├── ImagePreprocessor.kt # 图像预处理与量化
│       │   ├── MobilenetDemo.kt # 推理流程封装
│       │   ├── DemoAssetHelper.kt # assets 资源拷贝
│       │   └─ MainActivity.kt   # Compose UI
│       ├── jni/
│       │   ├── nnsdk_jni.cpp    # JNI 实现 (dlopen libnnsdk.so)
│       │   └─ nnsdk2.h          # NNSDK2 C API 头文件
│       └─ libs/arm64-v8a/      # 方案侧提供的 Native 库
│           ├── libadla.so
│           └─ libnnsdk.so
```

4.2.2 分层设计

层级	模块	职责
UI 层	MainActivity	图片选择、预览、触发推理、展示 Top-5 结果
业务层	MobilenetDemo	串联 init → preprocess → infer → postprocess 全流程
封装层	NnSdk2 / NnStruct	对外暴露 Kotlin API
JNI 层	nnsdk_jni.cpp	dlopen 加载 libnnsdk.so, RegisterNatives 注册本地方法
SDK 层	libnnsdk.so	NNSDK2 推理接口 (amlnn_init / run / outputs_get 等)
驱动层	libadla.so	ADLA NPU 驱动

调用链路：

```
MainActivity → MobilenetDemo → NnSdk2 → nnsdk_jni → libnnsdk.so → libadla.so → NPU
```

4.2.3 JNI 封装要点

参考工程采用 **动态加载** 方式接入 NNSDK，JNI 层不静态链接 libnnsdk.so，而是在运行时通过 dlopen / dlsym 解析符号。优点：

- JNI 工程仅编译 libnnsdk_jni.so，体积极小；
- libnnsdk.so / libadla.so 由方案侧提供，便于与 BSP 版本独立升级；
- 可在 JNI_OnLoad 中统一完成符号注册与错误处理。

库加载顺序（NnSdk2.kt）：

1. System.loadLibrary("adla") — 预加载 ADLA 驱动
2. System.loadLibrary("nnsdk_jni") — 加载 JNI 桥接库

JNI 注册方式：在 JNI_OnLoad 中通过 RegisterNatives 将 C++ 函数绑定到 NnSdk2 的 native* 方法，无需 Java_com_example_... 命名规则。

4.2.4 Kotlin API 一览

NnSdk2 对外暴露的推理接口如下：

方法	说明
initFromFile(modelPath, ...)	从文件路径加载 .adla 模型，返回 context 句柄
initFromBuffer(modelBuffer, ...)	从内存 buffer 加载模型

方法	说明
<code>query(context, queryCmd, info)</code>	查询 SDK 版本、输入输出属性等
<code>inputsSet(context, buffers, sizes, indices)</code>	设置输入 tensor (Direct <code>ByteBuffer</code>)
<code>run(context)</code>	执行推理
<code>outputsGet(context, index)</code>	获取输出, 返回 <code>FloatArray</code> (SDK 内部反量化)
<code>destroy(context)</code>	释放 context

注意: `outputsGet` 不传入预分配的 buffer, 而是由 NNSDK 在 `amlnn_outputs_get` 内部回填 `buf / size`, JNI 层再拷贝为 Java `FloatArray`。这是与部分 C++ Demo 一致的正确用法, 自行预分配 buffer 可能导致输出全零。

4.3 部署步骤

4.3.1 环境与版本确认

部署前确认设备侧 ADLA 驱动与 NNSDK 已就绪, 方法与 [2.3 环境确认](#) 相同:

```
# NNSDK 版本
strings /vendor/lib64/libnnsdk.so | grep NNSDK

# ADLA 中断 (推理时应有增长)
cat /proc/interrupts | grep adla
```

项	要求
Android 版本	Android R 及以上
ABI	<code>arm64-v8a</code> (参考工程仅打包此架构)
NNSDK	NNSDK2 v1.0.0 及以上
模型格式	<code>.adla</code> (端侧独立编译产物)

4.3.2 准备 Native 库

将方案侧提供的 so 放入工程的 `jniLibs` 目录:

```
app/src/main/libs/arm64-v8a/
├── libadla.so
├── libnnsdk.so
└── ...
```

`build.gradle.kts` 中通过 `sourceSets` 指定 `jniLibs` 路径:

```
sourceSets {
    getByName("main") {
        jniLibs.srcDirs("src/main/libs")
    }
}
ndk {
    abiFilters += "arm64-v8a"
}
```

4.3.3 编译 JNI 桥接库

app/CMakeLists.txt 编译 libnnsdk_jni.so:

```
add_library(nnsdk_jni SHARED src/main/jni/nnsdk_jni.cpp)
target_link_libraries(nnsdk_jni PRIVATE ${log-lib} dl)
```

build.gradle.kts 启用 CMake 外部构建:

```
externalNativeBuild {
    cmake {
        path = file("CMakeLists.txt")
    }
}
```

编译产物 libnnsdk_jni.so 随 APK 安装; 运行时由 dlopen("libnnsdk.so") 动态链接 NNSDK。

4.3.4 准备模型与测试资源

将模型、标签、测试图片放入 assets/demo/:

```
assets/demo/
├─ mobilenet_v2_1.0_224_quant.adla
├─ labels.txt
├─ cat_224x224.jpg
├─ dog_224x224.jpg
└─ fish_224x224.jpeg
```

应用启动时, 自动将 assets 拷贝到应用私有目录 (getExternalFilesDir(null)/demo/), 推理时使用绝对路径加载模型:

```
val modelPath = DemoAssetHelper.getModelFile(context).absolutePath
val contextHandle = NnSdk2.initFromFile(modelPath, backendType =
    NnSdk2.BACKEND_ADLA_NPU)
```

4.3.5 推理流程

参考 MobilenetDemo.kt, 完整推理分为以下步骤:

1. init	amlnn_init + query 输入/输出 tensor 属性
---------	------------------------------------

2. preprocess	加载图片 → resize 224×224 → RGB 量化
3. input_set	inputsSet(Direct ByteBuffer)
4. run	amlnn_run
5. output_get	outputsGet → FloatArray logits
6. postprocess	softmax → Top-K 分类
7. release	destroy(context)

示例代码：

```
// 1. 初始化
val handle = NnSdk2.initFromFile(modelPath, backendType =
NnSdk2.BACKEND_ADLA_NPU)

// 2. 查询输入属性
val attrBuffer = NnStruct.createTensorAttrBuffer(0)
NnSdk2.query(handle, NnSdk2.QUERY_INPUT_ATTR, attrBuffer)
val inAttr = NnStruct.readTensorAttr(attrBuffer)

// 3. 预处理
val inputBuffer = ImagePreprocessor.preprocessQuant(bitmap, inAttr.scale,
inAttr.zp, inAttr.type)

// 4. 推理
NnSdk2.inputsSet(handle, arrayOf(inputBuffer), intArrayOf(inAttr.nElems))
NnSdk2.run(handle)
val logits = NnSdk2.outputsGet(handle, 0)

// 5. 释放
NnSdk2.destroy(handle)
```

4.3.6 自主封装指南

客户可基于参考工程裁剪或扩展 JNI 层，建议遵循以下原则：

原则	说明
动态加载 NNSDK	使用 <code>dlopen</code> / <code>dlsym</code> ，避免静态链接导致版本耦合
Direct ByteBuffer 传输入	<code>inputsSet</code> 要求 Direct Buffer，普通 heap buffer 无效
输出由 SDK 回填	<code>outputsGet</code> 不设 buf，由 <code>amlnn_outputs_get</code> 内部分配并返回
结构体对齐	<code>query</code> 使用的 buffer 须通过 <code>nativeSizeOf</code> 获取大小，按 native 字节序读写
线程安全	单个 context 不建议多线程并发 <code>run</code> ；多模型场景各用独立 context

原则	说明
版本绑定	<code>nnsdk2.h</code> 、 <code>.ad1a</code> 模型、 <code>libnnsdk.so</code> 须与 BSP 编译器版本配套

4.3.7 编译与运行

```
# 编译 APK
cd 01_MobileNet
./gradlew assembleDebug

# 安装到设备
adb install -r app/build/outputs/apk/debug/app-debug.apk

# 查看推理日志
adb logcat -s MobilenetDemo NnSdk2 NnSdk2Jni
```

安装后打开 **MobileNet Demo** 应用：选择测试图片 → 点击「开始检测」→ 查看 Top-5 分类结果。详细日志可通过 `adb logcat` 获取。